

DSA with C LANGUAGE

DSA

1. What is DSA organized collection of raw facts.

2. Purpose of DSA

Increase efficiency & decrease storage

3. Classification of DSA

-Linear & Non Linear

Linear - Arrays, Linked List, Stacks, Queues

Non Linear - Tree, Graph

-Homogenous & Non Homogeneous

- Static & Dynamic

4. Operations -

Search,

Traverse,

Insert, Update, Delete,

Merge,

Sort

5. Algorithm - step by step written steps called as Algorithm for any problem

Algorithm Complexity

Analysis of algorithm to perform task in term of Time & Space

Space Complexity

Total Memory by algorithm

DSA with C LANGUAGE

Time Complexity

Total Time taken by algorithm

WE ARE USING ARRAY FOR ALL THE SEARCHING & SORTING

Arrays and Pointer Basics

What is an Array?

An array is a collection of elements stored in contiguous memory locations. All elements are of the same data type.

```
int arr[5] = {10, 20, 30, 40, 50};
```

arr is a pointer to the first element (arr[0])

You can access elements using index: arr[2] is 30

Static vs Dynamic Arrays

Feature	Static Array	Dynamic Array
Memory Allocation	Compile-time	Run-time
Size	Fixed	Can be changed (manually, via reallocation)
Declaration	<code>int arr[10];</code>	<code>int* arr = (int*)malloc(size * sizeof(int));</code>
Flexibility	Less flexible	More flexible

DSA with C LANGUAGE

Static Array Example

```
int arr[5]; // memory is allocated during compilation
```

```
#include <stdlib.h>
```

```
int *arr;
```

```
int size = 5;
```

Array Operations

1. Traverse (Print All Elements)

Loop through all elements:

```
for (int i = 0; i < size; i++) {  
    printf("%d ", arr[i]);  
}
```

2. Insert an Element

Static Array Insert (at index position)

```
void insert(int arr[], int *n, int pos, int value) {  
    for (int i = *n; i > pos; i--) {  
        arr[i] = arr[i - 1]; // Shift right  
    }  
    arr[pos] = value;  
    (*n)++; // increase size  
}
```

DSA with C LANGUAGE

Example:

```
int arr[10] = {10, 20, 30, 40};  
int n = 4;  
insert(arr, &n, 2, 25); // Insert 25 at index 2  
// arr becomes: 10, 20, 25, 30, 40
```

3. Delete an Element (by index)

```
void delete(int arr[], int *n, int pos) {  
    for (int i = pos; i < *n - 1; i++) {  
        arr[i] = arr[i + 1]; // Shift left  
    }  
    (*n)--; // reduce size  
}
```

Example:

```
int arr[5] = {10, 20, 30, 40};  
int n = 4;  
delete(arr, &n, 1); // Delete element at index 1  
// arr becomes: 10, 30, 40
```

4. Search for an Element

```
int search(int arr[], int n, int key) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == key) return i; // Return index if found  
    }  
    return -1; // Not found  
}
```

DSA with C LANGUAGE

Example:

```
int index = search(arr, n, 30); // returns 1 (if 30 is at index 1)
```

Pointers and Arrays

arr is actually a pointer to the first element.

*(arr + i) is equivalent to arr[i].

Example:

```
int arr[3] = {1, 2, 3};
```

```
printf("%d", *(arr + 1)); // Outputs: 2
```

-----Searching-----

Method to find element in data structure with location.

1. Linear Searching-

- Traverse from start till element not found,
- used for small amount data

Algorithm for Linear Search

Step 1 - Begin

2 - Set array arr [3] = {1,2,3}

3 - Set i = 0;

4 - Input searching item

5 - Repeat step 6 -7 while i<3 i< ar[size]

6 - If a[i] = item then

DSA with C LANGUAGE

Print "Item Found" & location = i;

Exit

7 - i= i+1;

8- if i>=3 then,

Print item "Not Found";

Time Complexity Linear search

- Base case $O(1)$

- Average case $O(n)$

- Worst case $O(n)$

Binary Searching-

The idea is to use Binary Search since the input array is sorted.

We maintain two pointers: left and right, and repeatedly check the middle element.

If the middle matches the target, we return its index. Otherwise, we narrow the search space by comparing the target with the middle value.

Algorithm for Binary Search

Step 1 - Begin

2 - Set arr [3] = {1,2,3}

3 - Set lower point =0, upper = 1, f =0;

4 - Input searching item - 2

5 - Repeat step 6 -8 while lower<= upper

6 - set mid = lower+up /2;

7 - if arr[mid] == item then

DSA with C LANGUAGE

set f =1 & break;

8- if arr[mid] < item then,

Set lower = mid +1

Else

Set upper = mid-1

9 - if f=1 then

printf "item found" location = mid

else

printf "item not found"

Time Complexity Binary Search

- Base case $O(1)$

- Average case $O(\log n)$

- Worst case $O(\log n)$

----- Sorting -----

Arrange element in array in particular order either ascending or descending

Types of Sorting

1. Bubble
2. Selection
3. Merge
4. Quick
5. Insertion

DSA with C LANGUAGE

----- Bubble Sort -----

Bubble Sort - Arrange n elements n element of array by placing big element on proper position

Compare big element to near element in Array

Algorithm

- 1. Begin**
- 2. Define array**
- 3. Compare 1st to 2nd element**
 - if 1st element > 2nd element then swap**
 - move to the next pair of element**
- 4. Continue compare and swapping until largest element found**
- 5. Repeat for all the unsorted element**
- 6. Exit**

----- Selection Sort -----

- Selection sort arrange N element of array by placing the smallest item in the proper position in case of ascending order arrangement

DSA with C LANGUAGE

Algorithm

1. Begin
2. Find smallest element
3. Swap the smallest no with the 1st number
4. Repeat for rest
5. Swap again with newly find element
6. Keep repeating
7. Done sorting
8. Exit

$a[] = \{ 2 \ 4 \ 7 \ 9 \ 5 \}$

----- Insertion Sort -----

- Arrange N element of Array by inserting particular item in a particular place such way that items are in sorted order

Algorithm

1. Begin
2. Start with 2nd element and comp with 1st
3. Compare and shift, If current element is smaller, shift larger to right
4. Place the current element in correct sorted position within shifted position
5. Move to the next unsorted element and repeat 3 & 4 until all element sorted
6. Exit

DSA with C LANGUAGE

-----QUICK SORT-----

QUICK SORT

Items separate the list of elements into 2 parts and sort each part recursively.

It use divide and conquer method.

Partition of list performed based on PIVOT element

List is divided into 2 partition such that all elements to the left of PIVOT are smaller

And all elements to right of PIVOT are greater

ALGORITHM

1. Begin
2. Select start element of array as PIVOT
3. Scan and find the SMALLEST element from right side of array
4. Exchange the both element
5. Scan & find the BIGGEST element from the left side of list and exchange
6. Repeat a process until all the element of Left side are SMALLER and right side elements are GREATER THAN PIVOT
7. Now 2 sublist are available, APPLY same process until all element of array are not sorted
8. Exit

DSA with C LANGUAGE

----- MERGE SORT-----

MERGE SORT

MERGE Sort is a sort that splits the items to be sorted into 2 groups recursively, sort each group and Merge then into a final sorted sequence

ALGORITHM

1. Begin
2. Divide all element in two pairs. If single item remain make it single group
3. Sort each of the pair
4. Merge two pairs into a single pairs
5. Sort the merge pair
6. Repeat the above step until all the elements of array are not sorted
7. Exit

DSA with C LANGUAGE

----- Linked List -----

Array

- 1 - It is collection of homogenous (similar) data type
- 2 - Array elements are stored in continuous memory location.
- 3 - Array work with static data structure -
- 4 - Array elements are independent to each other
- 5 - Array takes more time
e.g. - Insertion, Deletion etc.

Linked List

- 1 - Linked List is collection of node (data & address)
- 2 - Elements can be stored anywhere in the memory
- 3 - LL work with dynamic data structure
- 4 - LL elements are dependent to each other
- 5 - LL takes less time e.g. - Insertion, Deletion

Linked List

Is the linear collection of data structure where data are not store sequentially inside the computer memory

But they are link with each other by the help of address

DSA with C LANGUAGE

Types

- Singly Linked List
- Doubly Linked List
- Circular Linked List

- Singly Linked List

Linear collection of data item called node. Where each node divides into 2 parts

DATA - LINK

- Data part store data item
- Link part stores the address of the next node
- start with special pointer called start pointer and
- terminated with NULL pointer

We Understand Technology

Algorithm

- 1 - Insert at beginning singly LL
- 2 - Insert at ending singly LL
- 3 - Insert at any position LL
- 4 - Delete from beginning
- 5 - Delete from ending
- 6 - Delete from any position

DSA with C LANGUAGE

Insert at beginning

1. Begin
2. If free = NULL then print overflow
3. If not Input new item
4. New pointer <- free pointer
5. Free<-link[new]
6. data [new] <- item
7. link[new]<- start
8. start<-new
9. Exit

-- Doubly LL

Linear collection of data item called node. Where each node divides into 3 parts

Previous | Data | Next

Data part stores data item

Next part stores the address of next node

Previous part store the address of previous node

Start with the First Pointer and Ending with last pointer

Traverse in both way - Forward and Backward

Algorithm

1 - Insert at beginning

DSA with C LANGUAGE

2 - Insert at ending

3 - Insert at any position

4 - Delete from beginning

5 - Delete from ending

6 - Delete from any position

Insert at beginning

1. Begin
2. if first pointer = NULL then print overflow
3. if not Input new item
4. new pointer <- first pointer
5. first pointer <- next[first pointer]
6. data[new] <- item
7. next[new] <- first pointer
8. previous [new] <- Null
9. previous [first] <- new
10. first <- New
11. exit

We Understand Technology

Circular Linked List

Circular LL is the variation of singly and doubly LL where 1st node point to last node and last node point to first node

Note -

It is used when we want traversing of data no of times without reinitialized

The start pointer as well as we can visit all the nodes from any node

DSA with C LANGUAGE

Types

1. Circular Singly LL
2. Circular Doubly LL

1. Circular Singly LL

If a last node of SLL hold the address of the start node then its called Circular Singly LL

Algorithm

Insert item at begin	Delete
at ending	Delete
at any position	Delete

Travers the element of circular singly LL

1 Begin

2 $i = \text{str}$, $j = 0$

3 repeat step 4 - 7 while $j < 2$

4 print Data [i]

5 $i = \text{link} [i]$

6 if $i = \text{str}$ then

 set $j = j + 1$;

DSA with C LANGUAGE

STACK

- Linear collection of data item that follow LIFO (Last-In-First-Out)

Where insertion and deletion takes place on one end called TOP of the stack

NOTE -

PUSH - Insertion

POP - Deletion

- Stack follow LIFO Operation Last In First Out
- Insertion and Deletion takes place at one end
- Performs 2 operation PUSH and POP
- PUSH - insert element in stack
- POP - delete element in stack
- STACK doesn't have type
- Only one pointer is used Top of Stack

Implementation

With using Array

With using Linked List

DSA with C LANGUAGE

PUSH

ALGORITHM

```
1 Begin          2==3
2 if TOP = N then
    printf ("Overflow & Exit")
3 Input new Item
4 TOP = TOP + 1
5 Stack [Top] <- item
6 Exit
```

POP

```
1 Begin
2 If TOP = -1 then
    print underflow & exit
3 Set item <- stack [TOP]
4 TOP <- TOP - 1
5 Print item deleted
6 Exit
```

DSA with C LANGUAGE

QUEUE

- Linear Collection of Data item
- Follow FIFO operation First In First Out
- Insertion & Deletion takes place on opposite end
- Performs 2 operation REAR & FRONT end
- REAR - used to insert the element in the QUEUE
- FRONT- used to delete the element from the QUEUE
- General queue, Circular Que, Doubly Que
- Only two pointer is used REAR and FRONT

Insertion == Enqueue

Deletion == Dequeue

Insertion Algorithm

1. Begin
2. If $\text{Rear} = N-1$ then
 Print "Overflow & Exit",
- 3 Input New Value
- 4 $\text{Rear} \leftarrow \text{Rear} + 1$
- 5 $\text{Que}[\text{Rear}] \leftarrow \text{Item}$
- 6 Exit